

Доклад

Текст

Коровкин Никита Михайлович

Содержание

1	Доклад: Ошибки проверки вводимых данных — Инъекция E-mail	5
1.1	1. Введение	5
1.2	2. Техническая основа уязвимости	5
1.2.1	Механизм атаки	6
1.3	3. Векторы атак и примеры	6
1.3.1	Атака 1: Внедрение скрытой копии (BCC Injection)	6
1.3.2	Атака 2: Подмена тела письма и фишинг	7
1.4	4. Архитектурные примеры (Уязвимый и Безопасный код)	7
1.4.1	Пример на Python (Уязвимый подход)	7
1.4.2	Пример на TypeScript / Node.js (Безопасный подход)	8
1.5	5. Последствия эксплуатации	9
1.6	6. Методы предотвращения и защиты	9
1.7	7. Заключение	10
	Список литературы	11

Список иллюстраций

Список таблиц

Ниже представлен подробный технический доклад, посвященный уязвимости Email Injection (инъекция почтовых заголовков), механизмам ее работы, последствиям и методам защиты.

1 Доклад: Ошибки проверки вводимых данных – Инъекция E-mail

1.1 1. Введение

Инъекция E-mail (Email Injection) — это уязвимость безопасности веб-приложений, которая возникает, когда не прошедшие должную проверку или очистку пользовательские данные напрямую используются сервером для формирования почтовых сообщений.

Чаще всего эта уязвимость встречается в формах обратной связи, подписках на рассылки, запросах на сброс пароля и функциях «поделиться с другом». Злоумышленник использует эту брешь, чтобы модифицировать структуру отправляемого электронного письма, добавляя скрытых получателей, изменяя тему сообщения или подменяя его тело.

1.2 2. Техническая основа уязвимости

Чтобы понять, как работает инъекция, необходимо рассмотреть, как формируются электронные письма. Протокол SMTP (Simple Mail Transfer Protocol) и стандарт MIME используют обычный текст для описания структуры письма.

Письмо состоит из двух основных частей, разделенных пустой строкой:

1. **Заголовки (Headers):** To, From, Subject, Cc, Bcc, Content-Type и т.д.

2. Тело письма (Body): Само содержимое сообщения.

Ключевым элементом синтаксиса является разделитель строк — **CRLF** (Carriage Return + Line Feed), который в программировании обозначается как `\r\n` (в шестнадцатеричном виде: `0x0D 0x0A`). Каждая строка заголовка должна заканчиваться CRLF.

1.2.1 Механизм атаки

Если приложение берет данные из поля ввода (например, email отправителя) и вставляет их в заголовок `From` или `Reply-To` без проверки на наличие управляющих символов, злоумышленник может ввести строку, содержащую `\r\n`. Это заставит почтовый сервер интерпретировать текст после `\r\n` как новый заголовок.

1.3 3. Векторы атак и примеры

1.3.1 Атака 1: Внедрение скрытой копии (BCC Injection)

Цель — использовать сервер жертвы для рассылки спама, чтобы обойти спам-фильтры (так как письмо уходит с доверенного IP-адреса).

Уязвимый сценарий: Пользователь заполняет форму обратной связи.

- Поле *Sender Email*: `attacker@example.com`
- Ожидаемый заголовок: `From: attacker@example.com`

Эксплуатация: Злоумышленник вводит в поле *Sender Email*: `attacker@example.com\r\nBcc: spam-victim1@mail.com, spam-victim2@mail.com`

Сформированный сервером пакет:

`To: support@yourdomain.com`

`From: attacker@example.com`

Bcc: spam-victim1@mail.com, spam-victim2@mail.com

Subject: New Contact Form Submission

Message body...

Сервер послушно отправит копию письма на указанные спам-адреса.

1.3.2 Атака 2: Подмена тела письма и фишинг

Злоумышленник может использовать двойной перенос строки `\r\n\r\n`, чтобы досрочно завершить блок заголовков и начать собственное тело письма.

Ввод злоумышленника в поле темы: `Help\r\n\r\nCongratulations! You won a prize. Click here: [http://evil.com](http://evil.com)`

Результат: Оригинальное тело письма будет воспринято сервером как продолжение поддельного, или почтовый клиент проигнорирует его, отобразив только фишинговый текст злоумышленника.

1.4 4. Архитектурные примеры (Уязвимый и Безопасный код)

Рассмотрим механику уязвимости на примерах популярных бэкенд-технологий (Python и TypeScript).

1.4.1 Пример на Python (Уязвимый подход)

Уязвимость часто возникает при ручном формировании сырых SMTP-сообщений.

```
import smtplib
```

```
def send_feedback(user_email, user_message):
```

```
# УЯЗВИМОСТЬ: Ввод пользователя вставляется напрямую в структуру заголовков
headers = f"From: {user_email}\r\nTo: admin@mysite.com\r\nSubject: Feedback\r\n"
full_message = headers + user_message

server = smtplib.SMTP('localhost')
server.sendmail(user_email, ['admin@mysite.com'], full_message)
server.quit()
```

Если в `user_email` передать `test@test.com\r\nBcc: spam@spam.com`, сервер это проглотит.

1.4.2 Пример на TypeScript / Node.js (Безопасный подход)

Современные библиотеки, такие как `nodemailer`, по умолчанию защищают от внедрения CRLF, если использовать их API правильно (передавая параметры в объект, а не формируя строку вручную).

```
import * as nodemailer from 'nodemailer';

async function sendFeedback(userEmail: string, userMessage: string) {
    let transporter = nodemailer.createTransport({ /* config */ });

    // БЕЗОПАСНО: Библиотека сама экранирует спецсимволы в полях 'from', 'to', 'subject'
    let info = await transporter.sendMail({
        from: userEmail, // Защищено nodemailer от CRLF
        to: "admin@mysite.com",
        subject: "New Feedback",
        text: userMessage,
    });
}
```


1.5 5. Последствия эксплуатации

1. **Блокировка серверов (Blacklisting):** Если через ваш сервер будет отправлен спам, IP-адрес или домен почтового сервера (и приложения) быстро попадет в черные списки (Spamhaus, SURBL и др.). Легитимные письма перестанут доходить до пользователей.
2. **Потеря репутации:** Получатели спама увидят, что письмо пришло с официального домена вашей компании.
3. **Раскрытие информации:** В некоторых случаях злоумышленник может настроить пересылку ответов на свой адрес (Reply-To injection), перехватывая конфиденциальную переписку.

1.6 6. Методы предотвращения и защиты

Защита от Email Injection строится на принципе нулевого доверия к пользовательскому вводу.

- **Использование современных API (Ключевое правило):** Никогда не формируйте MIME-сообщения конкатенацией строк. Используйте встроенные классы и методы современных фреймворков и библиотек (например, `email.message.EmailMessage` в Python, `nodemailer` в Node.js), которые автоматически кодируют заголовки и вырезают CRLF-символы.
- **Валидация формата (Whitelisting):** Принимайте в полях email только те строки, которые строго соответствуют формату адреса электронной почты. Используйте надежные регулярные выражения. В валидном email не может быть пробелов или символов перевода строки.
- **Санитизация ввода:** Принудительно удаляйте или заменяйте символы `\r` (возврат каретки) и `\n` (перевод строки) из любых данных, которые могут попасть в почтовые заголовки (например, из поля “Тема”, если оно задается пользователем).

- **Отказ от передачи пользовательских email в заголовок From:** Практика отправки писем *от имени* пользователя (где From: user@gmail.com) часто нарушает политики DMARC/SPF/DKIM. Лучше использовать фиксированный адрес отправителя (например, noreply@yourdomain.com), а email пользователя помещать в заголовок Reply-To (после строгой валидации) или просто в тело письма.

1.7 7. Заключение

Инъекция E-mail — это классический пример уязвимости отсутствия валидации ввода, эксплуатирующий архитектуру протоколов, разработанных десятилетия назад. Несмотря на то, что современные фреймворки и библиотеки берут на себя большую часть работы по безопасной обработке заголовков, понимание механики CRLF-инъекций критически важно для проектирования безопасной архитектуры бэкенда и правильной настройки почтовых сервисов.

Список литературы